Working Title: Automated Reasoning over Substitutions in Agda

Marius Weidner
University of Freiburg, Germany

Abstract

Mechanizing the metatheory of programming languages in proof assistants is difficult because of recurring challenges with variable binding. Every formalization requires proving and correctly applying standard lemmas about substitutions such as those describing composition and interaction with binders. Historically researchers treated substitution as a meta level operation but explicit substitution calculi bring these operations into the syntax. The σ-calculus provides reduction rules that ensure confluence and strong normalization so that users can compare normal forms instead of applying lemmas manually.

Tools like Autosubst 2 provide this procedure via tactics to produce *propositional* equalities. We instead register σ-calculus rules as native reduction rules in Agda using its REWRITE mechanism to produce *definitional* equalities. Native embedding is challenging because it requires precise control over reduction behavior and care to maintain the logical consistency of Agda.

We treat renamings and substitutions separately at first in a scoped syntax setting which is safe and see that the concepts help a lot when working with intrinsically typed syntax: If substitution appears as an operation on the *index* of intrinsically typed syntax then rewriting the σ-calculus laws avoids transfer hell in subsequent substitution lemmas. Finally we investigate how to adapt the σ-calculus to a setting where substitutions and renamings are treated uniformly from a proof engineering perspective. This uniform treatment is less safe because more work is needed to verify the required properties of the resulting rewrite system.

2012 ACM Subject Classification Theory of computation → Equational logic and rewriting

Keywords and phrases Explicit Substitutions, Rewrite Systems, Agda

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Structure

The paper is organized into several sections. Section 2 introduces the theoretical background and tools. Section 3 presents an embedding of the σ-calculus as native reduction rules into Agda for multi-sorted, scoped syntax, treating renamings and substitutions separately. Section 4 shows how the rewriting approach prevents the transfer hell problem for complex intrinsically typed syntax. Section 5 explains how to adapt the laws to a setting where renamings and substitutions can be treated uniformly. Finally, we outline directions for future work in Section 6, discuss related work in Section 7 and conclude the paper in Section 8.

Contributions

The main contributions of this work are

- an embedding of the σ-calculus with first-class renamings into Agda, in a scoped and multi-sorted setting, using rewrite rules, exemplified with System F (Section 3);
- proof of the XXX laws for substitutions for expressions in intrinsically typed System F without entering transfer hell

© Marius Weidner;
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

- a novel version of the σ-calculus, where the laws are stated in a Kit- and Map-based setting. We effectively lift the distinction between variables and terms, as well as between renamings and substitutions, from the syntactic to the type level.

The supplement contains the complete Agda code underlying this paper.

2 Preliminaries

This section introduces the concepts and tools required for the developments in this work. We summarize

- Agda and language features relevant to our work: rewrite rules and opaque blocks;
- The σ_{SP}-calculus;
- Multi-sorted and scoped syntaxes in Agda.

2.1 Agda and Relevant Language Features

Agda is a dependently typed functional language and proof assistant based on Martin-Löf type theory [15], where proofs are verified via type checking. Proofs are significantly simplified when terms are *definitionally equal* (i.e. reduce to the same term). When terms are not definitionally equal, users must resort to manual equational reasoning using propositional equality, which is a cumbersome process as Agda lacks automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort.

Beyond standard features like indexed inductive types and dependent pattern matching [8], this work is based on two more Agda-specific language features.

Rewrite Rules

User-defined rewrite rules [9, 7] extend Agda's computation by treating propositional equalities as reduction rules. A function $\mathsf{eq} : (x_1 : A_1) \ldots \to \mathsf{f}\, p_1 \ldots p_n \equiv v$ can be registered as a rewrite rule if:

1. The head f is a postulate, defined function, or constructor with distinct variable type parameters.
2. Every variable x_i appears in a pattern position among $p_1, \ldots, p_n$[1].
3. The left side is neutral (it does not further reduce).

For example, the right identity of addition satisfies these criteria:

```
+-idʳ : ∀ n → n + 0 ≡ n
+-idʳ zero    = refl
+-idʳ (suc n) = cong suc (+-idʳ n)
```

Once registered via {-# REWRITE +-idʳ #-}, Agda treats $n + 0$ as definitionally equal to n:

```
_ : ∀ {n} → n + 0 ≡ n
_ = refl
```

Users must ensure rules are terminating and confluent to retain logical consistency. While Agda provides conservative confluence checks via `--confluence-check`, termination is not verified, but also not a critical property[9].

[1] The Agda documentation includes a precise definition for pattern positions: `https://agda.readthedocs.io/en/latest/language/rewriting.html`

Symbols

Variables x

innermost	zero
outer	suc x

Terms t

var	$`x$
abs	$\lambda.\, t$
app	$t_1\ t_2$
lookup	$x \mathbin{\&} \sigma$
subst	$t \cdots \sigma$

Substitutions σ

id	id
ext	$t \cdot \sigma$
comp	$\sigma_1 \mathbin{;} \sigma_2$
wk	wk
lift	$\uparrow\sigma \doteq ((`\mathsf{zero}) \cdot (\sigma \mathbin{;} \mathsf{wk}))$

Reduction

Definitional Laws

id	$x \mathbin{\&} \mathsf{id}$	$= `x$
$\mathsf{ext_Z}$	$\mathsf{zero} \mathbin{\&} (t \cdot \sigma)$	$= t$
$\mathsf{ext_S}$	$\mathsf{suc}\ x \mathbin{\&} (t \cdot \sigma)$	$= x \mathbin{\&} \sigma$
comp	$x \mathbin{\&} (\sigma_1 \mathbin{;} \sigma_2)$	$= (x \mathbin{\&} \sigma_1) \cdots \sigma_2$
wk	$x \mathbin{\&} \mathsf{wk}$	$= `(\mathsf{suc}\ x)$

Traversal Laws

var	$(`x) \cdots \sigma$	$= x \mathbin{\&} \sigma$
lam	$(\lambda.\, t) \cdots \sigma$	$= \lambda.(t \cdots (\uparrow\sigma))$
app	$(t_1\ t_2) \cdots \sigma$	$= (t_1 \cdots \sigma)\ (t_2 \cdots \sigma)$

Interaction Laws

$\mathsf{id_L}$	$\mathsf{id} \mathbin{;} \sigma$	$= \sigma$
$\mathsf{id_R}$	$\sigma \mathbin{;} \mathsf{id}$	$= \sigma$
assoc	$(\sigma_1 \mathbin{;} \sigma_2) \mathbin{;} \sigma_3$	$= \sigma_1 \mathbin{;} (\sigma_2 \mathbin{;} \sigma_3)$
dist	$(t \cdot \sigma_1) \mathbin{;} \sigma_2$	$= (t \cdots \sigma_2) \cdot (\sigma_1 \mathbin{;} \sigma_2)$
interact	$\mathsf{wk} \mathbin{;} (t \cdot \sigma)$	$= \sigma$
η-id	$(`\mathsf{zero}) \cdot \mathsf{wk}$	$= \mathsf{id}$
η-law	$(\mathsf{zero} \mathbin{\&} \sigma) \cdot (\mathsf{wk} \mathbin{;} \sigma)$	$= \sigma$

Monad Laws

r-id	$t \cdots \mathsf{id}$	$= t$
comp	$(t \cdots \sigma_1) \cdots \sigma_2$	$= t \cdots (\sigma_1 ; \sigma_2)$

Figure 1 The three-sorted σ-calculus

Opaque Blocks

Opaque definitions [13] treat code as non-unfolding symbols. Definitions within an opaque block are visible but treated abstractly during type checking:

```
opaque
  forty-two : ℕ
  forty-two = 42
```

Outside this block, forty-two does not reduce to 42 during equality checks:

```
_ : forty-two ≡ 42
_ = {!   !} -- not provable.
```

However, the unfolding keyword can be used to manually expose the underlying definition when needed:

```
opaque
  unfolding forty-two

  _ : forty-two ≡ 42
  _ = refl
```

2.2 The σ_{SP}-Caclulus

The σ-calculus [1] formalizes substitutions via explicit reduction rules. We utilize a three-sorted variant of the σ_{SP}-calculus [20], which provides a sound, confluent [10], and complete [20] foundation. Our instantiation for the λ-calculus is presented in Figure 1.

The syntax distinguishes between de Bruijn variables, terms, and parallel substitutions. Parallel substitutions map variables to terms and support composition. Unlike standard accounts, we include explicit syntax for variable usage, requiring distinct symbols for suc and wk to shift variables or substitutions, respectively. Applying wk to a term shifts all internal variables. We also define a *lifting* shorthand to push substitutions under binders; lifting preserves the bound variable while weakening the substitution's terms to maintain correct scoping.

Following [23], we categorize reduction rules into four groups:

- **Definitional**: Variable lookup in substitutions.

```
data Kind : Set where
  ⋆ : Kind

data Type (n : ℕ) : Set where
  ‘_        : Fin n → Type n
  ∀[α:_]_   : Type (suc n) → Type n
  _⇒_       : Type n → Type n → Type n

data Expr (n m : ℕ) : Set where
  ‘_    : Fin m → Expr n m
  λx_   : Expr n (suc m) → Expr n m
  Λα_   : Expr (suc n) m → Expr n m
  _·_   : Expr n m → Expr n m → Expr n m
  _•_   : Expr n m → Type n → Expr n m
```

```
data Sort : Set where
  expr type kind : Sort

Scope = List Sort
Scoped = Scope → Sort → Set

data _∋_ : Scoped where
  zero : ∀ {S s} → (s :: S) ∋ s
  suc  : ∀ {S s s′} → S ∋ s → (s′ :: S) ∋ s

data _⊢_ : Scoped where
  ‘_       : S ∋ s → S ⊢ s
  λx_      : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_      : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_  : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  _·_      : S ⊢ expr → S ⊢ expr → S ⊢ expr
  _•_      : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_      : S ⊢ type → S ⊢ type → S ⊢ type
  ⋆        : S ⊢ kind
```

Figure 2 Classically Scoped Syntax vs. Multi-Sorted and Scoped Syntax

- **Interaction**: Equalities between substitution expressions.
- **Traversal**: Propagation through terms and under binders.
- **Monad**: Behavior of substitutions applied to terms.

We omit object-language reductions (e.g., β-reduction) to focus strictly on substitution behavior. While renamings as first-class constructs will break completeness [24] later, we will prioritize a large space of solvable equations going forward. We maintain that our three-sorted variant remains terminating and confluent.

TODO: TALK ABOUT EXTENSION WITH FIRST CLASS RENAMINGS BECAUSE NOT STRUCTUALLY DEFINED

2.3 Multi-Sorted and Scoped Syntaxes in Agda

Unlike λ-calculus, many languages feature multiple syntactic categories. System F [11, 17], for instance, has expressions, types, and kinds. To prevent scoping errors, we utilize *intrinsically scoped syntax*, where inductive syntax definitions are indexed by free variables to ensure terms are correctly scoped by construction. Figure 2 (left) shows a standard implementation: types are indexed by type-variable counts, and expressions by both type and expression variable counts. Variables use de Bruijn indices (Fin n). This separate-datatype approach requires defining many substitution permutations (e.g., type-in-type, type-in-expr, expr-in-expr).

To avoid this combinatorial explosion, we define the uniform, multi-sorted syntax in Figure 2 (right). Terms form a single datatype ($S \vdash s$) indexed by a target sort s and a Scope S (a list of variable sorts). Variables are represented across multiple sorts simultaneously by membership proofs ($S \ni s$) that act as de Bruijn indices. This unified structure enables renamings and substitutions to operate uniformly across all sorts.

```
opaque
  _→ᴿ_ : Scope → Scope → Set
  S₁ →ᴿ S₂ = ∀ {s} → S₁ ∋ s → S₂ ∋ s

  wk : S →ᴿ (s :: S)
  wk x = suc x
  -- omitted: idᴿ _&ᴿ_ _·ᴿ_
  --          _;ᴿᴿ_ _↑ᴿ_ _···ᴿ_
_→ˢ_ : Scope → Scope → Set
S₁ →ˢ S₂ = ∀ {s} → S₁ ∋ s → S₂ ⊢ s

_&ˢ_ : S₁ ∋ s → (S₁ →ˢ S₂) → S₂ ⊢ s
x &ˢ σ = σ x

idˢ : S →ˢ S
idˢ = `_

_·ˢ_ : S₂ ⊢ s → (S₁ →ˢ S₂) →
  (s :: S₁) →ˢ S₂
(t ·ˢ _) zero    = t
(_ ·ˢ σ) (suc x) = σ x
```

```
-- omitted: _;ᴿˢ_ _;ˢᴿ_ ...

-- Lifting shorthand (not opaque)
_↑ˢ_ : (S₁ →ˢ S₂) → ∀ s → ((s :: S₁) →ˢ (s :: S₂))
(σ ↑ˢ _) = (` zero) ·ˢ (σ ;ˢᴿ wk)

-- Traversal Laws (not opaque)
_···ˢ_ : S₁ ⊢ s → (S₁ →ˢ S₂) → S₂ ⊢ s
(` x)        ···ˢ σ = (x &ˢ σ)
(λx e)       ···ˢ σ = λx (e ···ˢ (σ ↑ˢ _))
(Λα e)       ···ˢ σ = Λα (e ···ˢ (σ ↑ˢ _))
(∀[α: k ] t) ···ˢ σ = ∀[α: k ···ˢ σ ] (t ···ˢ (σ ↑ˢ _))
(e₁ · e₂)    ···ˢ σ = (e₁ ···ˢ σ) · (e₂ ···ˢ σ)
(e ▪ t)      ···ˢ σ = (e ···ˢ σ) ▪ (t ···ˢ σ)
(t₁ ⇒ t₂)    ···ˢ σ = (t₁ ···ˢ σ) ⇒ (t₂ ···ˢ σ)
⋆            ···ˢ σ = ⋆

opaque
  unfolding _→ˢ_
  _;ˢˢ_ : (S₁ →ˢ S₂) → (S₂ →ˢ S₃) → (S₁ →ˢ S₃)
  (σ₁ ;ˢˢ σ₂) x = (σ₁ x) ···ˢ σ₂
```

Figure 3 Scoped and Multi-Sorted Renaming and Substitution (excerpt)

3 The Scoped Sigma Calculus with First-Class Renamings in Agda

This section presents a direct embedding of the σ-calculus rules with first-class renamings, following the conceptual theory underlying AUTOSUBST 2, as native reduction rules in Agda using its rewrite mechanism. We use the multi-sorted and scoped version of SystemF from Figure 2 as an example for any custom syntax. The approach described in this section directly translates to any multi-sorted and scoped syntax definition.

3.1 Scoped and Multi-Sorted Renaming and Substitution

We define renamings ($_\to^R_$) and substitutions ($_\to^S_$) uniformly for all variable sorts by indexing them over scopes. A renaming from scope S_1 to S_2 maps each variable in S_1 to a variable of the same sort in S_2. Substitutions map variables to terms of the corresponding sort.

We first define renaming primitives like identity (id^R), weakening (wk^R), and extension ($_\cdot^R_$). Substitution primitives mirror these but use the superscript S. Weakening remains strictly a renaming. We choose the definition order carefully for structural recursion. We use weakening to define the lifting of substitutions ($_\uparrow^S_$).

We provide four composition primitives to support every combination of renamings and substitutions. These include renaming with renaming ($_;^{RR}_$), renaming with substitution ($_;^{RS}_$), substitution with renaming ($_;^{SR}_$), and substitution with substitution ($_;^{SS}_$). We keep these separate even though the first two can be defined uniformly.

Traversal functions ($_\cdots^R_$ and $_\cdots^S_$) are defined by structural induction on the syntax. Each clause applies a transformation to a term and lifts the operation when traversing binders. These clauses match the σ-calculus laws. We want these functions to reduce during type checking.

```
  -- Definitional Laws
  id^R : x &^R id^R                 ≡ x
  wk   : x &^R wk {s = s'}          ≡ suc x
  ext_0^R : zero &^R (x ·^R ρ)      ≡ x
  ext_s^R : (suc x') &^R (x ·^R ρ)  ≡ x' &^R ρ

  comp^RR : x &^R (ρ1 ;^RR ρ2) ≡ (x &^R ρ1) &^R ρ2
  comp^SS : x &^S (σ1 ;^SS σ2) ≡ (x &^S σ1) ⋯^S σ2
  -- omitted: id^S ext_0^S ext_s^S
  --          comp^RS comp^SR
-- Monad Laws
right-id^R : t ⋯^R id^R ≡ t
right-id^S : t ⋯^S id^S ≡ t

compositionality^RR : (t ⋯^R ρ1) ⋯^R ρ2 ≡ t ⋯^R (ρ1 ;^RR ρ2)
compositionality^RS : (t ⋯^R ρ1) ⋯^S σ2 ≡ t ⋯^S (ρ1 ;^RS σ2)
compositionality^SR : (t ⋯^S σ1) ⋯^R ρ2 ≡ t ⋯^S (σ1 ;^SR ρ2)
compositionality^SS : (t ⋯^S σ1) ⋯^S σ2 ≡ t ⋯^S (σ1 ;^SS σ2)
  -- Coincidence Laws
  coincidence       : t ⋯^S (ρ ;^RS id^S) ≡ t ⋯^R ρ
  coincidence-fold^V : t ⋯^S ((` x) ·^S (ρ ;^RS id^S)) ≡
                       t ⋯^R (x ·^R ρ)
  coincidence-fold^T : t ⋯^S ((t' ⋯^R ρ) ·^S (ρ ;^RS id^S)) ≡
                       (t ⋯^S (t' ·^S id^S)) ⋯^R ρ
  -- Interaction Laws
  -- omitted.
```

```
{-# REWRITE
  id^R wk ext_0^R ext_s^R
  id^S ext_0^S ext_s^S
  comp^RR comp^RS comp^SR comp^SS

  comp-id_l^RR comp-id_l^RS comp-id_r^RR
  comp-id_r^SS comp-id_r^SR comp-id_l^SS
  associativity^RRR associativity^RRS
  associativity^RSR associativity^RSS
  associativity^SRR associativity^SRS
  associativity^SSR associativity^SSS
  distributivity^RR distributivity^RS
  distributivity^SR distributivity^SS
  interact^R interact^S
  η-id^R η-id^S η-law^R η-law^S

  coincidence
  coincidence-fold^V coincidence-fold^T

  right-id^R right-id^S
  compositionality^RR compositionality^RS
  compositionality^SR compositionality^SS
#-}
```

Figure 4 The Scoped and Multi-Sorted σ-calculus with First-Class Renamings

Renaming and substitution primitives are declared opaque to block their reduction. This abstraction is cruical for rewriting σ-calculus laws where primitives must act as constant symbols. Blocking reduction makes rewrite rules predictable. We also declare the types opaque to prevent misuse as functions. We use explicit application syntax (_&^{R}_ and _&^{S}_) to allow rewriting on renaming or substitution applied to a variable.

3.2 The Sigma Calculus with First-Class Renamings

We could restrict renamings to the definition of substitutions. However reasoning about them directly is often useful for instance in the context morphism lemma[12]. We treat renamings as first class constructs similar to Autosubst 2. Figure4 presents the σ-calculus laws with first class renamings.

Definitional laws now involve the primitives for both renamings and substitutions. Many **Interaction laws** share a similar shape. For example we require eight associativity laws, for all combinations of the four composition operators. **Traversal laws** are given via the traversal definitions in the previous section. Most laws are proven independently of the concrete syntax, except **Monad laws**, which must be shown by induction over the syntax. We highlight these syntax dependent laws in red throughout this work.

Coincidence laws emerge from the interaction between renamings and substitutions[24].

They allow us to recover renamings from their embedding as substitutions. Extracting arbitrary renamings from deeply nested substitutions would require a dedicated solving strategy, which we leave for future work but consider important for a more complete account. In her PhD thesis,[23] proves the coincidence law by induction over the syntax. We found the law is not syntax-dependent when the monad laws are available:

$$
\begin{array}{ll}
t \cdots^{S} (\rho \,;^{RS}\, \mathsf{id}^{S}) & \equiv\langle\ \mathsf{sym}\ (\mathsf{compositionality}^{RS}\ t)\ \rangle \\
(t \cdots^{R} \rho) \cdots^{S} \mathsf{id}^{S} & \equiv\langle\ \mathsf{right\text{-}id}^{S}\ (t \cdots^{R} \rho)\ \rangle \\
t \cdots^{R} \rho & \square
\end{array}
$$

The remaining coincidence laws follow analogously.

The proofs of the other laws are standard and we refer to the supplement, blue laws are proven syntax independently while red laws require structural induction. To this end, we unfold the definitions that are declared opaque. Inside the proofs we make use function extensionality which is admissable to the theory underlying Agda.

3.3 Discussion

The laws presented in Section 3.2 are an extension of the classic σ-calculus rules to a setting with renamings. They fulfill the requirements to be registered as rewrite rules without further modification. This is a consequence of marking the relevant symbols opaque. Inside the proofs of the laws themselves, the definitions can still unfold when needed, while externally they behave as uninterpreted symbols and are reduced only by the σ-calculus laws. Separating renamings and substitutions causes some duplication. Traversal and monad laws must be defined twice. Every subsequent result also needs to be proven for both cases.

This construction reveals a broader concept that serves as a general recipe for integrating algebraic laws into the proof assistants.

- Define symbols for some structure.
- Establish confluent rewrite rules that describe custom algebraic laws.
- Instantiate the structure with an underlying model to prove the laws.
- Hide the reductions of the model to preserve confluence and symbolic behavior.

The structure is instantiated to prove correctness but the underlying reductions need to be hidden from the outside. This abstraction prevents the system from introducing unwanted simplifications during type checking. For example lookup should stay as a symbol instead of reducing to function application. Similarly wk should remain symbolic instead of reducing to suc.

This concept of Dedukti [5] style constants and rewrite rules with hidden models could be applied to other structures like natural numbers, lists or sets. Such an approach would provide more definitional equalities for indexed types like vectors. To this end, Agda would benefit greatly from an extension with rewriting modulo commutativity to support more confluent systems. But then there is also more work needed to ensure the confluence checker accepts more complex systems and handles opaque definitions differently.

4 Avoiding Transfer Hell in Complex Intrinsically Typed Syntax

Rewriting σ-calculus laws improves intrinsically typed syntax in addition to scoped syntax. We demonstrate this by reformulating System F as an example for any complex syntax, where substitution appears on indices, and prove the monad laws for expression substitution. These proofs are simplified significantly if the σ-calculus laws for types are rewritten first. This development represents a significant step toward using intrinsically typed syntax over

complex languages for proofs in Agda. Previous works like [19] required thousands of lines of code to prove substitution laws. The length of these proofs stems from the transfer lemma subst appearing inside the *signature* of expression substitution lemmas. This leads to the infamous transfer hell where users must manually manage shifting transfers. Our rewriting approach mitigates this issue.

4.1 Substitutions for Complex Intriniscally Typed Syntax

Types in System F are well formed by construction so a scoped syntax is sufficient. We restrict the syntax from section 2.3 to the necessary sorts kind type : Sort. We then apply the definitions for substitutions and rewrite rules from section 3. Following the approach of [6] we provide an intrinsically typed definition for System F expressions.

```
data Ctx : Scope → Set where
  ∅   : Ctx []
  _,_ : Ctx S → S ⊢ type → Ctx S
  _,⋆ : Ctx S → Ctx (type :: S)

data _∈_ : S ⊢ type → Ctx S → Set where
  zero : t ∈ (Γ , t)
  suc  : t ∈ Γ → t ∈ (Γ , t')
  suc⋆ : t ∈ Γ → (weaken t) ∈ (Γ ,⋆)

data Expr {S : Scope} (Γ : Ctx S) : S ⊢ type → Set where
  `_   : t ∈ Γ →
         Expr Γ t
  λx_  : Expr (Γ , t₁) t₂ →
         Expr Γ (t₁ ⇒ t₂)
  Λα_  : {t : (type :: S) ⊢ type} → Expr (Γ ,⋆) t →
         Expr Γ (∀[α: k ] t)
  _·_  : Expr Γ (t₁ ⇒ t₂) → Expr Γ t₁ →
         Expr Γ t₂
  _·_  : {t : (type :: S) ⊢ type} →
         Expr Γ (∀[α: k ] t) → (t' : S ⊢ type) →
         Expr Γ (t [ t' ])
```

The Ctx stores well scoped types and includes the _,⋆ constructor for explicit type weakening. Well typed variables _∈_ similarly include explicit type weakening via suc⋆. Expressions are indexed over their respective types and utilize the shorthands weaken $t = t \cdots^R$ wk and $t\,[\,t'\,] = t \cdots^S (t' \cdot^S \mathsf{id}^S)$ on types. For a more detailed explanation of these underlying definitions we refer to [6].

Similar to [6] we define expression substitutions indexed by type substitutions and renamings follow an analogous pattern.

```
_→ᴱˢ[_]_ : Ctx S₁ → (S₁ →ˢ S₂) → Ctx S₂ → Set
Γ₁ →ᴱˢ[ σ ] Γ₂ = ∀ t → t ∈ Γ₁ → Expr Γ₂ (t ⋯ˢ σ)
```

In constrast to [6] we can state all renaming and substitution lemmas without using subst inside function bodies. Composition of expression substitution serves as an example.[2]

```
_;ˢˢᴱ_ : (Γ₁ →ᴱˢ[ σ₁ ] Γ₂) → (Γ₂ →ᴱˢ[ σ₂ ] Γ₃) →
  (Γ₁ →ᴱˢ[ σ₁ ;ˢˢ σ₂ ] Γ₃)
(σᴱ₁ ;ˢˢᴱ σᴱ₂) _ x = (σᴱ₁ _ x) ⋯ᴱˢ[ _ ] σᴱ₂
```

Here the traversal $_\cdots^{SE}[_]_$ applies a type substitution (inside the brakets) and expression substitution to a term simultaneously. Inside the body agda applies the compositionality^{SS} for type substitutions automatically to coerce the *Agda type* of the expression $(\sigma^E{}_1\ _\ x)\ \cdots^{SE}[\ _\ ]\ \sigma^E{}_2$ from Expr Γ $((t \cdots^S \sigma_1) \cdots^S \sigma_2)$ to Expr Γ $(t \cdots^S (\sigma_1 ;^{SS} \sigma_2))$.

[2] You may compare our version to the version found in [19] at `https://github.com/proglang/SystemF/blob/e56afb91bfc8dc314bdec6de39a44799fe7e52fa/src/StratF/ExprSubstitution.agda#L144`

4.2 Avoiding Transfer Hell in Proofs of Expression Substitution Laws

Even more, we state, as an example, the category laws of expression substitutions, without subst in the signature and thereby avoiding transfer hell:

Note that subst would have been required on the left of the equality to coerce the index of the expression substitution using compositionality^{SS}[3]

4.3 Discussion

Rewriting σ-calculus laws on the indices of intrinsically typed syntax simplifies substitution lemmas further down the index hiearchy.

Expression substitution σ-calculus laws could also be registered as rewrite rules. This approach might provide subject reduction almost for free through structural induction. A prototype of this method exists although it is omitted for brevity. Confluence for the expression substitution rewrite system remains unverified even though the property likely holds.

5 Sigma Calculus Rules for a Uniform Treatment of Renamings and Substitutions

RECENT TREND TO DEFINE REN AND SUB UNIFORMELY CITE.. INVESTIGATE SIGMA LAWS IN THIS CONTEXT

6 Further Work

7 Related Work

Explicit Substitution Calculi

There exist a variety of explicit substitution calculi, but we focus on the original σ-calculus [1], which includes identity, extension, shifting, and composition primitives, and the σ_{SP}-calculus [20], which adds further reductions to ensure confluence. Only the σ_{SP}-calculus is both sound and complete, enabling all equations between terms to be derived via reduction. Both completeness and convergence are established through mechanized proofs [20, 23].

Substitution Libraries for Proof Assistants

[14] in section 7.1 similarly to our embed the autosubst 2 laws (sigma calc with first-class renamings) into Rocq using local rewrite rules. they do give a model for renamings and substitutions as functions, was we do, but they are not able to prove the laws given the underlying model (or even have a mechanism to do so).

We have already discussed Autosubst 2 [24, 21, 23] which forms the direct theoretical foundation of our work.

There exists another Agda library with a similar goal to Agdasubst, called Abstract Binding Trees, used for instance by [22]. It permits constructing an object language from a generic syntax, similar to the approach in **??**, but it supports only extrinsic scoping and single-sorted syntaxes. The library provides abstractions over arbitrary predicates about the

[3] Compare this to the version found in [19]: `https://github.com/proglang/SystemF/blob/e56afb91bfc8dc314bdec6de39a44799fe7e52fa/src/StratF/ExprSubstFusion.agda#L95`

object language, rather than focusing on specific relations such as typing [18]. The library also offers incomplete experimental support for rewriting using σ-calculus laws, but only handles renamings and substitutions separately, similar to the approach in Section 3.2, rather than in a uniform treatment.

More distantly related is Nominal 2 [25], an Isabelle package that addresses the α-equivalence problem not via de Bruijn indices, but using nominal datatypes and freshness conditions. Their approach is closer to the intuitive notion of variables, but the tool does not provide any strategy for solving equations involving terms and substitutions.

Abstractions over Variables/Terms and Renamings/Substitutions

We already discussed the original work on Kits [16] and their adaptation to multi-sorted and scoped syntax [18] throughout Section 5.

The intrinsically scoped version of this approach also appears in recent work, for example in a paper written by [4], where a uniform treatment of renaming and substitution is explored in an intrinsically typed setting. Their work also includes a structure and definitions similar to the structural kit order we defined in **??**. They additionally unify variables and terms, which we deliberately do not do, as variables are library-defined while terms are user-supplied. Furthermore they hide the structual dependence of renamings and substitutions in the kit structure itself, using terminating checking directly, to make Agda see this dependence. The approach by saffrich instead hides it behind instead hide it in instance resolution. A further application of the Kit approach is given by [26], who use it to reason about metatheory in the context of linear languages.

Another line of research concerns generic programming approaches [2, 3], which allow the construction of syntax trees from general building blocks supporting sums, products, and binder introduction. The generic programming approach also allows for a generic structure that abstracts not only over renamings and substitutions, but also over semantics and other relevant language structures. The generic syntax approach can be adapted to a scoped and multi-sorted setting [18]. Importantly, generic programming and the Agdasubst library are orthogonal, as shown in **??**.

8 Conclusion

WE WOULD LOVE INTEGRATED CONFLUENCE CHECKING IN AGDA ALSO COMMUTATIVITY REWRITING AND THE CHECK ONLY LASTED 0.16s, SO AGDA SHOULD BE ABLE TO DO SO INTERERESTING: HOW TO CHECK CONFLUENCE IN PRESENCE OF OPAQUE (UNFOLDING MAY BREAK IT)!!! VERIFY SIGMA CALC WITH FIRST CLASS RENAMINGS IS CONFLUENT ADD MORE COINCIDENCES We have shown how to take advantage of the σ-calculus laws to automatically solve equations inside proofs about syntaxes with binding in Agda, similar to Autosubst 2 in Rocq.

We first embedded the σ-calculus with first-class renamings directly into Agda as reduction rules, using its rewrite mechanism to yield definitional equalities for a formalization of System F.

We then wanted to unify proofs about renamings and substitutions, which led to the current state of the Agdasubst library. To achieve our goal, we introduced the notions of Kits and Maps from prior work and adapted the σ-calculus laws to a Kit and Map setting.

We demonstrated how Agdasubst can be used to prove properties involving substitutions for formal languages, including subject reduction for System F with subtyping using a single

substitution-preserves-typing lemma, and provided an instantiation of a scoped universe of syntaxes with binding.

The Agdasubst library does not constitute a stable release candidate yet. We have outlined potential directions for future development and plan to continue improving the library over time.

References

1. M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, page 31–46, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/96709.96712.
2. Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type and scope safe universe of syntaxes with binding: Their semantics and proofs, 2021. URL: https://arxiv.org/abs/2001.11001, arXiv:2001.11001.
3. Guillaume Allais, James Chapman, Conor McBride, and James McKinna. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, CPP 2017, page 195–207, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3018610.3018613.
4. Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. Substitution without copy and paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)*, Birmingham, UK, 2025.
5. Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a logical framework based on the $\lambda\pi$-calculus modulo theory, 2023. URL: https://arxiv.org/abs/2311.07185, arXiv:2311.07185.
6. James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. System f in agda, for fun and profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings*, page 255–297, Berlin, Heidelberg, 2019. Springer-Verlag. doi:10.1007/978-3-030-33636-3_10.
7. Jesper Cockx. Type Theory Unchained: Extending Agda with Userined Rewrite Rules. In Marc Bezem and Assia Mahboubi, editors, *25th International Conference on Types for Proofs and Programs (TYPES 2019)*, volume 175 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1--2:27, Dagstuhl, Germany, 2020. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2019.2, doi:10.4230/LIPIcs.TYPES.2019.2.
8. Jesper Cockx and Andreas Abel. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi:10.1145/3236770.
9. Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi:10.1145/3434341.
10. Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM*, 43(2):362–397, March 1996. doi:10.1145/226643.226675.
11. Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Phd thesis, Éditeur inconnu, 1972. URL: https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf.
12. Healfdene Goguen and James McKinna. Candidates for substitution. *LFCS report series-Laboratory for Foundations of Computer Science ECS LFCS*, 1997. URL: https://www.lfcs.inf.ed.ac.uk/reports/97/ECS-LFCS-97-358/.

13 Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420*, 2022. URL: https://arxiv.org/abs/2210.05420.

14 Yann Leray and Théo Winterhalter. Encode the cake and eat it too: Controlling computation in type theory, locally. *Proc. ACM Program. Lang.*, 10(POPL), January 2026. doi:10.1145/3776704.

15 Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73--118. Elsevier, 1975. URL: https://www.sciencedirect.com/science/article/pii/S0049237X08719451, doi:10.1016/S0049-237X(08)71945-1.

16 Conor McBride. Type-preserving renaming and substitution. Unpublished manuscript, 2005. URL: http://strictlypositive.org/ren-sub.pdf.

17 John C. Reynolds. Towards a theory of type structure. In B. Robinet, editor, *Programming Symposium*, pages 408--425, Berlin, Heidelberg, 1974. Springer Berlin Heidelberg. URL: https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf.

18 Hannes Saffrich. Abstractions for Multi-Sorted Substitutions. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 32:1--32:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.32, doi:10.4230/LIPIcs.ITP.2024.32.

19 Hannes Saffrich, Peter Thiemann, and Marius Weidner. Intrinsically typed syntax, a logical relation, and the scourge of the transfer lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development*, TyDe 2024, page 2–15, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3678000.3678201.

20 Steven Schäfer, Gert Smolka, and Tobias Tebbi. Completeness and decidability of de bruijn substitution algebra in coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs*, CPP '15, page 67–73, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2676724.2693163.

21 Steven Schäfer, Tobias Tebbi, and Gert Smolka. Autosubst: Reasoning with de bruijn terms and parallel substitutions. In Christian Urban and Xingyuan Zhang, editors, *Interactive Theorem Proving*, pages 359--374, Cham, 2015. Springer International Publishing. URL: https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf.

22 Jeremy G. Siek and Tianyu Chen. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming*, 31:e30, 2021. doi:10.1017/S0956796821000241.

23 Kathrin Stark. *Mechanising Syntax with Binders in Coq*. PhD thesis, Saarland University, Saarbrücken, Germany, January 2020. Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.). URL: https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf.

24 Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: reasoning with multi-sorted de bruijn terms and vector substitutions. CPP 2019, page 166–180, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3293880.3294101.

25 Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. Nominal 2. *Archive of Formal Proofs*, February 2013. https://isa-afp.org/entries/Nominal2.html, Formal proof development.

26 James Wood and Robert Atkey. A linear algebra approach to linear metatheory. *Electronic Proceedings in Theoretical Computer Science*, 353:195–212, December 2021. URL: http://dx.doi.org/10.4204/EPTCS.353.10, doi:10.4204/eptcs.353.10.